МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра инструментального и прикладного
программного обеспечения (ИиППО)

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ
Аналитическая практика

приказ Университета о направлении на практику от «31» августа 2026 г. № 7110-С

Отчет представлен к
рассмотрению:
Студент группы ИКБО-18-25 «12» сентября 2026 г. Курилов А.И.
(подпись и расшифровка подписи)

Отчет утвержден.
Допущен к защите:

Руководитель практики
от кафедры «12» сентября 2026 г. Благирев М.М.
(подпись и расшифровка подписи)

Руководитель практики
от профильной организации «12» сентября 2026 г. Жиркова М.И.
(подпись и расшифровка подписи)

Москва 2026 г.

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра инструментального и прикладного
программного обеспечения (ИиППО)

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ
Аналитическая практика

Студенту 2 курса учебной группы ИКБО-18-25
Курилову Андрею Игоревичу

Место и время практики: Общество с ограниченной ответственностью «Бюджетные и Финансовые Технологии», с 01 сентября 2026 г. по 12 сентября 2026 г.

Должность на практике: программист

1. СОДЕРЖАНИЕ ПРАКТИКИ:

1.1. Изучить: состав и принципы работы информационных технологий и программных средств, применяемых в организации, в том числе отечественного производства, а также источники по теме практики, включая издания на иностранном языке

1.2. Практически выполнить: выполнить декомпозицию поставленной задачи на подзадачи; провести поиск и анализ информации по теме, оценив достоверность источников; обосновать выбор инструментальных средств для решения задачи; систематизировать результаты и оформить отчёт с источниками литературы.

1.3. Ознакомиться: с требованиями информационной безопасности при работе с данными и сведениями организации, правилами разграничения доступа и резервного копирования, а также правилами библиографического описания источников

2. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ: нет

3. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: в соответствии с методическими указаниями по учебной практике по направлению 09.03.04 «Программная инженерия»

Руководитель практики от кафедры
«01» сентября 2026 г. _____ (Благирев М.М.)
Подпись

Руководитель практики от профильной организации
«01» сентября 2026 г. _____ (Жиркова М.И.)
Подпись

Задание получил
«01» сентября 2026 г. _____ (Курилов А.И.)
Подпись

СОГЛАСОВАНО:
Заведующий кафедрой:

«01» сентября 2026 г. _____ (Болбаков Р.Г.)
Подпись

Проведенные инструктажи:

Охрана труда: «01» сентября 2026 г.
Инструктирующий _____ Жиркова М.И., директор департамента обеспечения качества
Подпись
Инструктируемый _____ Курилов А.И.
Подпись

Техника безопасности: «01» сентября 2026 г.
Инструктирующий _____ Жиркова М.И., директор департамента обеспечения качества
Подпись
Инструктируемый _____ Курилов А.И.
Подпись

Пожарная безопасность: «01» сентября 2026 г.
Инструктирующий _____ Жиркова М.И., директор департамента обеспечения качества
Подпись
Инструктируемый _____ Курилов А.И.
Подпись

С правилами внутреннего распорядка ознакомлен: «01» сентября 2026 г.
_____ Курилов А.И.
Подпись

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

РАБОЧИЙ ГРАФИК
ПРОВЕДЕНИЯ АНАЛИТИЧЕСКОЙ ПРАКТИКИ

студента Курилова А.И. 2 курса группы ИКБО-18-25 очной формы обучения, обучающегося по направлению подготовки 09.03.04 Программная инженерия профиль «Разработка программных продуктов и проектирование информационных систем»

Неделя	Сроки выполнения	Этап	Отметка о выполнении
1	01.09.2026	Подготовительный этап, включающий в себя организационное собрание (Вводная лекция о порядке организации и прохождения учебной практики, инструктаж по технике безопасности, получение задания на практику)	
1-2	01.09.2026-07.09.2026	Декомпозиция задачи, поиск и анализ материала по теме практики с оценкой достоверности источников, обоснование выбора инструментальных средств, структурирование материала	
2	08.09.2026	Представление руководителю практики структурированного материала	
2	08.09.2026-11.09.2026	Подготовка окончательной версии отчета по практике (Оформление материалов отчета в полном соответствии с требованиями на оформление письменных учебных работ студентов)	
2	12.09.2026	Представление руководителю практики окончательной версии отчета по практике, загруженной в СДО	

Содержание практики и планируемые результаты согласованы с руководителем практики от профильной организации

Руководитель практики от кафедры _____/Благирев М.М., ассистент/

Руководитель практики от профильной организации _____/Жиркова М.И., директор департамента обеспечения качества/

Обучающийся _____/Курилов А.И./

Согласовано:

Заведующий кафедрой _____/Болбаков Р.Г., к.т.н./

РЕФЕРАТ

Отчет 24 с., 19 листингов, 1 табл., 13 ист.

ОБЗОР ВОЗМОЖНОСТЕЙ СТАТИЧЕСКОЙ ТИПИЗАЦИИ В ЯЗЫКЕ PYTHON

PYTHON, СТАТИЧЕСКАЯ ТИПИЗАЦИЯ, АННОТАЦИИ ТИПОВ, TYPING, MYPY, PYRIGHT, PYDANTIC, NUMPY

Объектом исследования являются система типов Python и инструменты статического анализа.

Цель работы – изучить возможности статической типизации и её влияние на надёжность и сопровождаемость программного кода.

В работе рассмотрены динамическая и статическая типизация, аннотации типов, возможности модуля typing, а также инструменты mypy и pyright. В практической части выполнено сравнение кода с аннотациями и без них, исследованы TypedDict, Generic, Pydantic v2 и numpy.typing.

Установлено, что статическая типизация позволяет выявлять часть ошибок до запуска программы, улучшает структуру и сопровождаемость кода. При этом она не заменяет тестирование и проверку данных во время выполнения.

Результаты работы могут применяться при разработке и сопровождении Python-приложений для повышения их надёжности и качества.

СОДЕРЖАНИЕ

РЕФЕРАТ 5

СОДЕРЖАНИЕ 6

ВВЕДЕНИЕ 7

1 Декомпозиция задачи и теоретические основы статической типизации в Python 8

1.1 Анализ информационных технологий и эволюция подходов к типизации 8

1.2 Обзор встроенных и продвинутых средств модуля typing 9

Объединение типов и необязательные значения. 9

Обобщённые типы 10

Протоколы 11

Типизированные словари 11

1.3 Сравнительный анализ инструментальных средств статического анализа 12

2 Практическое исследование и экспериментальный анализ 15

2.1 Сравнительный анализ кода до и после внедрения аннотаций типов 15

2.2 Валидация данных во время выполнения с использованием Pydantic v2 18

2.3 Типизация числовых массивов NumPy 19

2.4 Систематизация результатов анализа и формирование выводов 20

Заключение 22

Список использованных источников 23

ВВЕДЕНИЕ

Python – язык с динамической типизацией: тип значения определяется во время выполнения программы, и заранее указывать его для каждой переменной не требуется. Такой подход ускоряет написание кода и делает его гибким, но с ростом проекта усложняет контроль корректности данных.

Для небольших программ это обычно не проблема. В крупных проектах количество функций, классов и связей между ними растёт, поэтому ошибки в типах данных нередко обнаруживаются только во время выполнения. Это усложняет тестирование, сопровождение и переписывание кода.

Решить проблему помогают аннотации типов и инструменты статического анализа. Они позволяют заранее описать данные, которые принимает и возвращает функция, и проверить это до запуска программы. Python при этом остаётся динамическим языком: аннотации не обязательны и сами по себе не меняют порядок выполнения кода.

Цель работы: рассмотреть возможности статической типизации в Python и оценить её роль в повышении надёжности и сопровождаемости кода.

Для достижения поставленной цели нужно решить следующие задачи:

1. Рассмотреть развитие типизации в Python, принцип утиной типизации и появление стандарта PEP 484.
2. Изучить основные возможности модуля typing, включая объединения типов, обобщённые типы, протоколы и типизированные словари.
3. Сравнить статические анализаторы mypy и pyright и определить особенности их применения.
4. Провести практический анализ статической проверки типов и проверки данных во время выполнения при обнаружении ошибок.

Объект исследования: система типов и инструменты статического анализа языка программирования Python.

Предмет исследования: способы описания типов данных и выявления потенциальных ошибок с помощью статической типизации, а также их применение на практических примерах.

1 Декомпозиция задачи и теоретические основы статической типизации в Python

1.1 Анализ информационных технологий и эволюция подходов к типизации

На ранних этапах развития Python использовалась исключительно динамическая типизация: переменная не закрепляется за типом на этапе написания программы, и одна и та же переменная может в разное время хранить число, строку или любой другой объект.

Один из ключевых принципов Python: утиная типизация. При работе с объектом важна не его принадлежность к конкретному классу, а наличие нужных методов и свойств [1, 2]. Если объект их предоставляет, его можно использовать в соответствующем участке программы.

Функция может полагаться на наличие у объекта нужного метода, не требуя принадлежности к конкретному классу. Это делает код гибким и позволяет использовать в одном контексте разные объекты.

Однако у динамической типизации есть недостатки: ошибки в типах данных обнаруживаются только во время выполнения программы, а не заранее; в больших проектах сложнее понять, какие данные ожидает функция, а изменения в коде становятся более рискованными; автодополнение и проверки в среде разработки работают слабее, поэтому часть ответственности за типы ложится на тестирование.

С ростом программных систем эти проблемы становятся заметнее. В ответ на них в Python появился механизм опциональной статической типизации: в 2014 году была принята спецификация PEP (Python Enhancement Proposal – предложение по улучшению Python) 484, определившая аннотации типов [3, 4].

Аннотации позволяют явно указать ожидаемые типы параметров функций, возвращаемых значений и переменных, что показано в листинге 1.

Листинг 1 – Пример функции с аннотациями типов

```
def calculate_total(price: float, quantity: int) -> float:
    return price * quantity
```

В этом примере price должен быть числом с дробной частью, quantity должен быть целым числом, а результат работы функции тоже должен быть числом с дробной частью.

Сама по себе такая запись не мешает передать функции неправильное значение. Интерпретатор Python по умолчанию не проверяет соответствие аннотациям. Их задача в том, чтобы снабдить информацией инструменты статического анализа и разработчика.

Если, например, вместо quantity передать строку, статический анализатор обнаружит потенциальную ошибку ещё до запуска программы. Это особенно ценно в крупных проектах, где вручную отследить все связи между функциями практически невозможно.

Так Python перешёл от исключительно динамической модели к гибридному подходу, при котором разработчик может писать код без аннотаций, а где нужно, подключать статическую проверку для отдельных функций, модулей или всего проекта.

1.2 Обзор встроенных и продвинутых средств модуля typing

Модуль typing из стандартной библиотеки расширяет возможности описания типов [5]. Он нужен, когда простого int, str или float уже недостаточно.

Объединение типов и необязательные значения.

Функция иногда принимает значения разных типов. Для этого случая существует объединение типов, пример использования которого приведён в листинге 2.

Листинг 2 – Объединение типов с использованием Union и оператора |

```
from typing import Union

def stringify(value: Union[int, str]) -> str:
    return str(value)

def stringify(value: int | str) -> str:
    return str(value)
```

Функция stringify принимает либо целое число, либо строку. При этом результат всегда преобразуется в строку. А начиная с Python 3.10 вместо Union[int, str] можно использовать более короткую запись с помощью оператора | (вертикальная черта).

Другой распространённый случай: параметр, который может содержать значение или None. Для него используется Optional, как показано в листинге 3.

Листинг 3 – Использование Optional для необязательного значения

```
from typing import Optional

def find_user(user_id: int) -> Optional[dict]:
    ...
```

Здесь указано, что функция вернёт либо словарь с данными пользователя, либо None, если пользователь не найден. Аннотация сразу предупреждает разработчика, что результат нужно обрабатывать в двух вариантах [4, 6].

Обобщённые типы

Обобщённые типы позволяют создавать функции и классы, которые сохраняют информацию о типе содержащихся в них данных [6].

Например, можно создать стек, работающий с элементами произвольного типа, как показано в листинге 4.

Листинг 4 – Обобщённый класс Stack с параметром типа

```
from typing import TypeVar, Generic

T = TypeVar("T")

class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()
```

Здесь T обозначает тип, который определяется при использовании класса: для Stack[int] анализатор будет считать, что стек хранит только целые числа, и попытка добавить туда строку станет ошибкой типов. Главное

преимущество такого подхода в том, что один и тот же класс подходит для разных типов данных, но информация о типе элементов не теряется.

Протоколы

Протоколы реализуют структурную типизацию и позволяют формально описывать принцип утиной типизации, что показано в листинге 5 [2, 7].

Листинг 5 – Описание протокола для структурной типизации

```
from typing import Protocol

class Comparable(Protocol):
    def __lt__(self, other: object) -> bool:
        ...

def sort_items(items: list[Comparable]) -> list[Comparable]:
    return sorted(items)
```

В этом случае функции не важен конкретный класс объекта: достаточно, чтобы он поддерживал сравнение через метод __lt__, а значит, можно использовать разные классы, если они реализуют нужный набор методов. Статический анализатор при этом проверяет, соответствует ли объект требованиям протокола, так что Protocol сохраняет гибкость утиной типизации, но делает эти требования явными и проверяемыми.

Типизированные словари

TypedDict применяется для описания словарей, имеющих заранее определённый набор ключей и типов значений, как показано в листинге 6 [8].

Листинг 6 – Описание структуры словаря с помощью TypedDict

```
from typing import TypedDict

class UserOrder(TypedDict):
    user_id: int
    product: str
    quantity: int
```

Пример описывает структуру заказа: словарь должен содержать идентификатор пользователя, название товара и количество, и каждому ключу соответствует свой тип. Это особенно полезно при работе с данными из JSON: в обычном словаре разработчику приходится самому помнить, какие ключи там должны быть, а TypedDict описывает эту структуру заранее и даёт анализатору возможность проверять обращения к её полям.

Каждое из этих средств решает свою задачу: объединения описывают допустимые варианты типов, обобщения дают универсальные структуры, протоколы задают требования к поведению объектов, а TypedDict описывает структуру словарей. Вместе они позволяют подробно описывать данные даже в сложных программах.

1.3 Сравнительный анализ инструментальных средств статического анализа

Для анализа кода нужны отдельные инструменты, и среди них для Python чаще всего используют mypy и pyright.

Мypy – статический анализатор типов, созданный при участии автора спецификации PEP 484, распространяется как отдельная утилита командной строки и широко применяется для проверки соответствия кода указанным типам [2, 6]. В нём можно настраивать уровень строгости проверки, что позволяет постепенно внедрять типизацию в уже существующий проект, не аннотируя сразу весь код.

Pyright – статический анализатор типов от Microsoft, реализованный на TypeScript и тесно интегрированный с Visual Studio Code через расширение Pylance [6, 9]. Его главное преимущество: скорость. Инкрементальная проверка анализирует только изменённые участки проекта, поэтому разработчик быстрее получает информацию о найденных проблемах.

Основные различия инструментов представлены в таблице 1.

Таблица 1 – Сравнительный анализ mypy и pyright

Критерий	**mypy**	**pyright**
Скорость работы	Средняя: полный анализ проекта может занимать больше времени	Высокая: эффективная инкрементальная проверка
Строгость проверки	Настраиваемая, по умолчанию менее строгая	Строгая проверка и развитые средства анализа
Поддержка IDE	Интеграция через плагины и расширения	Тесная интеграция с Visual Studio Code через Pylance
Сужение типов	Поддерживается	Расширенная поддержка сложных условных конструкций
Язык реализации	Python	TypeScript
Обновления	Регулярные релизы	Частые обновления

Важная возможность статических анализаторов: сужение типа. Оно означает уточнение возможного типа переменной после проверки условия.

Например, если функция принимает число или строку, внутри неё можно проверить тип значения, как показано в листинге 7.

Листинг 7 – Сужение типа с помощью проверки isinstance

```
def process(value: int | str) -> None:
    if isinstance(value, int):
        print(value + 1)
    else:
        print(value.upper())
```

До проверки value может быть числом или строкой. После isinstance(value, int) анализатор понимает, что в первой ветви находится число и сложение допустимо, а во второй остаётся строка, для которой доступен метод upper(). Это значит, что сужение типов учитывает не только исходные аннотации, но и логику программы, и проверка становится точнее.

Выбор между mypy и pyright зависит от задач проекта: mypy подходит там, где нужна гибко настраиваемая проверка типов, в том числе как часть автоматических проверок кода, а pyright удобнее при разработке в Visual Studio Code, где важна быстрая обратная связь. При этом оба инструмента решают одну задачу: используют информацию из аннотаций для поиска ошибок ещё до запуска программы [10].

2 Практическое исследование и экспериментальный анализ

2.1 Сравнительный анализ кода до и после внедрения аннотаций типов

Возьмём функцию, которая считает стоимость заказа. Данные в неё обычно приходят из веб-формы, и до момента вызова никто не гарантирует, каким типом обернётся, скажем, количество товара. Исходный вариант функции и пример входных данных приведены в листинге 8.

Листинг 8 – Исходная реализация обработки заказа без аннотаций типов

```
def process_order(order):
    total = order["price"] * order["quantity"]
    if order["discount"]:
        total -= total * order["discount"]
    return round(total, 2)


orders = [
    {"price": 199.99, "quantity": 3, "discount": 0.1},
    {"price": 49.5, "quantity": "2", "discount": None},
]

for order in orders:
    print(process_order(order))

539.97
Traceback (most recent call last):
  File "order.py", line 14, in <module>
    print(process_order(order))
  File "order.py", line 2, in process_order
    total = order["price"] * order["quantity"]
TypeError: can't multiply sequence by non-int of type 'float'
```

Первый заказ считается верно. Второй вызывает сбой в программе: количество пришло строкой из формы, а Python не умеет умножать дробное число на строку. Ошибка всплывает только на этом конкретном заказе и только во время выполнения – ни сигнатура функции, ни её тело ничего не сообщают о том, что нужно число.

Перепишем через типизированный словарь, как показано в листинге 9 [8].

Листинг 9 – Реализация обработки заказа с использованием TypedDict

```
from typing import TypedDict


class Order(TypedDict):
    price: float
    quantity: int
    discount: float | None


def process_order(order: Order) -> float:
    total = order["price"] * order["quantity"]
    if order["discount"]:
        total -= total * order["discount"]
    return round(total, 2)


orders: list[Order] = [
    {"price": 199.99, "quantity": 3, "discount": 0.1},
    {"price": 49.5, "quantity": "2", "discount": None},
]
```

Для проверки кода достаточно запустить mypy для данного файла. Результат проверки приведён в листинге 10.

Листинг 10 – Результат статической проверки типизированного заказа с помощью mypy

```
$ mypy order.py
order.py:19: error: Incompatible types (expression has type "str",
TypedDict item "quantity" has type "int")  [typeddict-item]
Found 1 error in 1 file (checked 1 source file)
```

Муру сразу указывает строку и причину: второй заказ не соответствует структуре Order. Программу запускать не пришлось.

Похожая по происхождению ошибка возникает, когда функция обращается к словарю как к объекту, то есть путает доступ по ключу с доступом через точку. Обобщённый тип помогает отловить и её на уровне хранилища, как показано в листинге 11.

Листинг 11 – Обобщённое хранилище Repository и проверка доступа к данным

```
from typing import Generic, TypeVar

T = TypeVar("T")


class Repository(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []

    def add(self, item: T) -> None:
        self._items.append(item)

    def get(self, index: int) -> T:
        return self._items[index]


orders_repo: Repository[Order] = Repository()
orders_repo.add({"price": 199.99, "quantity": 3, "discount": None})

first_order = orders_repo.get(0)
print(first_order.price)  # обращение как к атрибуту
```

Результат проверки этого кода с помощью mypy приведён в листинге 12.

Листинг 12 – Результат проверки Repository с помощью mypy

```
$ mypy repo.py
repo.py:27: error: "Order" has no attribute "price"  [attr-defined]
Found 1 error in 1 file (checked 1 source file)
```

Repository помнит, элементы какого типа в нём лежат, поэтому анализатор знает: get() вернёт Order, а у Order нет атрибута price, есть только ключ price в словаре. Без Generic эта путаница обнаружилась бы только в момент вызова.

Тот же файл проверяет и pyright. Результат приведён в листинге 13.

Листинг 13 – Результат проверки Repository с помощью pyright

```
$ pyright repo.py
repo.py:27:19 - error: Cannot access attribute "price" for class "Order"
    Attribute "price" is unknown
    Use ["price"] to reference item in TypedDict
(reportAttributeAccessIssue)
1 error, 0 warnings, 0 informations
```

Ошибка та же, но формулировка отличается: pyright не просто указывает на отсутствие атрибута, а сразу подсказывает правильный синтаксис обращения к полю. На практике это совпадает с тем, что было сказано в разделе 1.3: оба анализатора решают одну задачу поиска ошибок типов до

запуска программы, но отличаются деталями диагностики и тем, насколько она удобна для разработчика [10].

2.2 Валидация данных во время выполнения с использованием Pydantic v2

TypedDict и аннотации – это лишь соглашение между разработчиком и анализатором. Во время выполнения программы они ничего не проверяют: если данные пришли из внешнего источника, функция примет словарь любой формы. Чтобы выявлять некорректные данные в момент их поступления, нужна отдельная библиотека, например Pydantic второй версии [11, 12]. Пример модели с ограничениями приведён в листинге 14.

Листинг 14 – Модель OrderModel для валидации данных с использованием Pydantic v2

```
from pydantic import BaseModel, Field, ValidationError


class OrderModel(BaseModel):
    price: float = Field(gt=0)
    quantity: int = Field(gt=0)
    discount: float | None = Field(default=None, ge=0, le=1)
```

С корректными данными модель отрабатывает так, как показано в листинге 15.

Листинг 15 – Валидация корректных данных с помощью Pydantic

```
data = {"price": 199.99, "quantity": "3", "discount": 0.1}
order = OrderModel(**data)
print(order)
price=199.99 quantity=3 discount=0.1
```

Строка "3" в поле quantity сама превратилась в целое число, т.к. Pydantic допускает такое приведение на входе, в отличие от статического анализатора, которому нужно точное совпадение типа.

С некорректными данными картина другая: результат проверки приведён в листинге 16.

Листинг 16 – Обнаружение некорректных данных с помощью Pydantic

```
bad_data = {"price": -10, "quantity": 0, "discount": 1.5}

try:
    OrderModel(**bad_data)
except ValidationError as e:
    print(e)
```

```
3 validation errors for OrderModel
price
  Input should be greater than 0 [type=greater_than, input_value=-10,
input_type=int]
quantity
  Input should be greater than 0 [type=greater_than, input_value=0,
input_type=int]
discount
  Input should be less than or equal to 1 [type=less_than_equal,
input_value=1.5, input_type=float]
```

Исключение ValidationError сразу перечисляет все нарушения, а не только первое найденное. Получается разделение труда: mypy ловит ошибки в структуре кода до запуска, Pydantic – ошибки в конкретных данных уже после запуска. Одно не заменяет другое. Отдельный практический плюс: ядро валидации в этой версии библиотеки переписано на Rust, и по сравнению с первой версией Pydantic проверка стала заметно быстрее.

2.3 Типизация числовых массивов NumPy

Интересный частный случай – типизация массивов NumPy, основной библиотеки для вычислений с числами в Python [13]. Функция ниже считает выручку по массиву цен и массиву количеств. Её реализация и пример использования приведены в листинге 17.

Листинг 17 – Типизация числовых массивов NumPy

```
import numpy as np
from numpy.typing import NDArray


def total_revenue(
    prices: NDArray[np.float64],
    quantities: NDArray[np.int64],
) -> np.float64:
    return np.sum(prices * quantities)


prices = np.array([199.99, 49.5, 15.0], dtype=np.float64)
quantities = np.array([3, 2, 1], dtype=np.int64)
print(total_revenue(prices, quantities))
713.97
```

Здесь тип аргумента описывает не просто массив, а массив чисел конкретной точности. Но у такой типизации есть слабое место: она проверяет тип элементов, а не длину массива. Если в массиве количеств случайно не хватает одной позиции, статический анализатор не обнаруживает проблему, что показано в листинге 18.

Листинг 18 – Обнаружение ошибки размеров массивов при выполнении программы

```
quantities = np.array([3, 2], dtype=np.int64)  # забыли одну позицию
print(total_revenue(prices, quantities))

$ mypy revenue.py
Success: no issues found in 1 source file
```

А при запуске программа падает, что видно в листинге 19.

Листинг 19 – Сбой работы программы при запуске

```
Traceback (most recent call last):
  File "revenue.py", line 11, in <module>
    print(total_revenue(prices, quantities))
  File "revenue.py", line 6, in total_revenue
    return np.sum(prices * quantities)
ValueError: operands could not be broadcast together with shapes (3,) (2,)
```

Массивы разной длины нельзя перемножить поэлементно, и NumPy сообщает об этом в терминах формы (shapes), а не типа. Муру эту ошибку не видит в принципе: несовпадение размеров массивов типом не описывается, и статический анализ здесь бессилен.

2.4 Систематизация результатов анализа и формирование выводов

Практика подтвердила теоретические выводы: использование TypedDict и Generic позволило выявить ошибки еще на этапе статического анализа с помощью mypy и pyright (в списке заказов и при неверном обращении к полям словаря), не доводя их до сбоя при запуске программы. Pydantic выполнил роль второго контура проверки, контролируя уже входные данные в процессе работы и при сбое выдавая весь список нарушений, а не одну причину.

Противоположным примером является NumPy. Механизм типизации здесь описывает только тип элементов массива, но не контролирует их количество, поэтому несовпадение длин массивов при статическом анализе не

отслеживается. Отсюда следует главный практический вывод: аннотации и статический анализ закрывают конкретный, довольно узкий класс ошибок, а не ошибки вообще [10]. Они не могут заменить проверку данных в реальном времени и стандартное тестирование, а служат важным дополнением к ним.

ЗАКЛЮЧЕНИЕ

Статическая типизация в Python не отменяет динамическую природу языка, а встраивается в неё как дополнительный слой. Разработчик сам решает, где нужны аннотации типов и насколько строгой должна быть их проверка. Такой компромисс сохраняет гибкость языка и одновременно поднимает предсказуемость кода в больших проектах: именно ответом на проблемы масштабирования и стало появление PEP 484 в 2014 году.

Модуль typing даёт для этого достаточный набор средств. Объединения типов, обобщения, протоколы и TypedDict решают разные задачи, но вместе позволяют точнее описывать данные и дают статическим анализаторам больше материала для проверки. При сравнении mypy и pyright выяснилось, что цель у инструментов одна, а устроены они по-разному. Муру делает акцент на гибкой настройке проверки, а pyright ставит превыше всего скорость анализа и плотную интеграцию с редактором.

В целом статическая типизация не отменяет необходимость тестирования, но помогает находить часть ошибок ещё до запуска программы, упрощает понимание кода и делает изменения в проекте менее рискованными. В этом и есть её главная практическая ценность.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Что такое утиная типизация (duck typing) в Python [Электронный ресурс] // Python School. – Режим доступа: https://python-school.ru/blog/python/ducl-typing-examples/ (дата обращения: 01.09.2026).
2. Протоколы в Python: утиная типизация по-новому [Электронный ресурс] // Habr. – Режим доступа: https://habr.com/ru/articles/557898/ (дата обращения: 01.09.2026).
3. Van Rossum, G. PEP 484 – Type Hints [Электронный ресурс] / G. van Rossum, J. Lehtosalo, Ł. Langa // Python.org. – 2014. – Режим доступа: https://peps.python.org/pep-0484/ (дата обращения: 02.09.2026).
4. Введение в аннотации типов Python [Электронный ресурс] // Habr: блог компании Lamoda. – Режим доступа: https://habr.com/ru/companies/lamoda/articles/432656/ (дата обращения: 02.09.2026).
5. Как устроена типизация в Python и зачем она нужна [Электронный ресурс] // Skillfactory Media. – Режим доступа: https://blog.skillfactory.ru/kak-ustroena-tipizatsiya-v-python/ (дата обращения: 03.09.2026).
6. Погружение в typing: как сделать Python почти типизированным [Электронный ресурс] // Habr. – Режим доступа: https://habr.com/ru/articles/982932/ (дата обращения: 03.09.2026).
7. Протоколы в Python [Электронный ресурс] // Habr: блог компании Wunder Fund. – Режим доступа: https://habr.com/ru/companies/wunderfund/articles/751424/ (дата обращения: 04.09.2026).
8. TypedDict – mypy 2.3.1 documentation [Электронный ресурс] // Read the Docs. – Режим доступа: https://mypy.readthedocs.io/en/stable/typed_dict.html (дата обращения: 04.09.2026).
9. Pyright documentation [Электронный ресурс] // Microsoft. – Режим доступа: https://microsoft.github.io/pyright/ (дата обращения: 05.09.2026).

10. Su, C. How Well Static Type Checkers Work with Gradual Typing? A Case Study on Python [Электронный ресурс] / C. Su, Y. Guo [и др.] // IEEE Xplore: International Conference on Program Comprehension (ICPC 2023). – 2023. – Режим доступа: https://ieeexplore.ieee.org/document/10173949 (дата обращения: 05.09.2026).

11. Pydantic documentation [Электронный ресурс] // Pydantic. – Режим доступа: https://docs.pydantic.dev/latest/ (дата обращения: 06.09.2026).

12. Pydantic 2: полное руководство для Python-разработчиков – от основ до продвинутых техник [Электронный ресурс] // Habr: блог компании Amvera. – Режим доступа: https://habr.com/ru/companies/amvera/articles/851642/ (дата обращения: 06.09.2026).

13. Type hints (numpy.typing) [Электронный ресурс] // NumPy documentation. – Режим доступа: https://numpy.org/doc/stable/reference/typing.html (дата обращения: 07.09.2026).